Regularized regression

Miguel García Moreno

2026-03-05

Abstract

This project fits a multiple linear regression for the response against the remaining covariates, diagnoses extreme multicollinearity, and applies elastic-net regularization with glmnet over a grid of alpha values. The best model is chosen by cross-validation using the one-standard-error rule.

Contents

1 PACKAGE INSTALLATION **3**

2 DATA EXTRACTION AND PREPROCESSING **4**

3 DATA SET PREPARATION **4**

4 MULTIPLE LINEAR MODEL **4**

4.1 MODEL CONSTRUCTION 4

4.2 RESIDUAL ANALYSIS 5

4.3 MULTICOLLINEARITY ANALYSIS 6

5 REGULARIZATION MODEL **7**

5.1 MODEL CONSTRUCTION 7

5.2 MODEL SELECTION 7

1 PACKAGE INSTALLATION

```
packages <- c('glmnet','Rcpp','foreach','survival','shape','utils','methods','Matrix','car')
for (i in 1:length(packages))
{
  if(require(packages[i],character.only = TRUE)) cat(packages[i], " loaded successfully\n")
  else {
    cat("Trying to install", packages[i], "\n")
    install.packages(packages[i])
    if(require(packages[i],character.only = TRUE)) cat(packages[i], " loaded successfully\n")
    else {
      stop("Could not install ", packages[i])
    }
  }
}
```

```
## Cargando paquete requerido: glmnet

## Warning: package 'glmnet' was built under R version 4.5.2

## Cargando paquete requerido: Matrix

## Loaded glmnet 4.1-10

## Cargando paquete requerido: Rcpp

## Warning: package 'Rcpp' was built under R version 4.5.2

## Cargando paquete requerido: foreach

## Warning: package 'foreach' was built under R version 4.5.2

## Cargando paquete requerido: survival

## Cargando paquete requerido: shape

## Warning: package 'shape' was built under R version 4.5.2

## Cargando paquete requerido: car

## Warning: package 'car' was built under R version 4.5.2

## Cargando paquete requerido: carData

## Warning: package 'carData' was built under R version 4.5.2
```

2 DATA EXTRACTION AND PREPROCESSING

```
load("AEM_03_Regularization_data.RData")
```

3 DATA SET PREPARATION

Removal of 6 variables according to the ID number. If the ID ends in "abcd", remove 6 variables among x1a, x2a, x3a, x1b, x2b, x4b, x1c, x3c, x4c, x2d, x3d, x4d

```
a=9
b=3
c=5
d=6
elim=c(19,29,39,13,23,43)
datos=datos[,-elim]
```

4 MULTIPLE LINEAR MODEL

Obtain the multiple regression model of variable Yj against the remaining variables. Analyse the multicollinearity problem.

4.1 MODEL CONSTRUCTION

```
lm.fit <- lm(Y~.,data=datos)
summary(lm.fit)
```

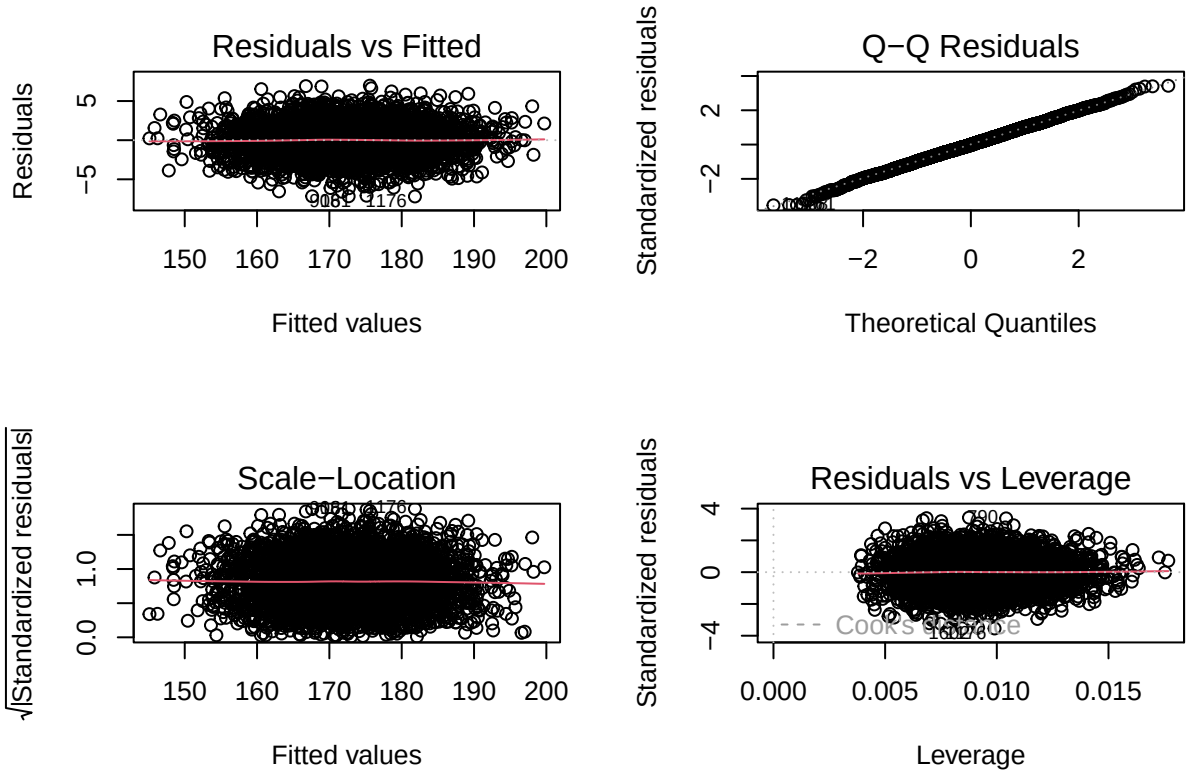
```
##
## Call:
## lm(formula = Y ~ ., data = datos)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -7.1906 -1.3227 -0.0383  1.3740  6.9666
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.04941    2.04637   0.513 0.608109
## X1           0.43819    0.94817   0.462 0.644000
## X2           3.27573    1.28337   2.552 0.010734 *
## X3          -1.03867    0.48925  -2.123 0.033816 *
## X4          -1.70244    2.14688  -0.793 0.427834
## X5          -0.57503    1.57022  -0.366 0.714228
## X6           4.32409    2.02848   2.132 0.033093 *
## X7           3.39626    2.96263   1.146 0.251712
## X8          -2.85097    3.11955  -0.914 0.360822
## X9           1.84331    0.77388   2.382 0.017270 *
## X10         -0.42443    0.95664  -0.444 0.657305
```

```
## X11          0.37020    0.14775   2.506 0.012264 *
## X13          0.70524    0.21379   3.299 0.000980 ***
## X14          0.56513    0.39066   1.447 0.148086
## X15         -0.96783    0.15580  -6.212 5.78e-10 ***
## X16          0.50785    0.23447   2.166 0.030377 *
## X17          1.55863    0.16786   9.285  < 2e-16 ***
## X19          0.23522    0.19882   1.183 0.236845
## X20          1.65928    0.18325   9.055  < 2e-16 ***
## X21          0.64370    0.16865   3.817 0.000137 ***
## X23          1.95065    0.23716   8.225 2.62e-16 ***
## X24         -2.80728    0.16348 -17.172  < 2e-16 ***
## X25         -1.57187    0.25757  -6.103 1.14e-09 ***
## X26         -1.98593    0.18606 -10.674  < 2e-16 ***
## X27         -1.94060    0.29300  -6.623 3.99e-11 ***
## X29         -1.90099    0.39887  -4.766 1.95e-06 ***
## X30          4.86427    0.20935  23.235  < 2e-16 ***
## X31          0.50336    0.16203   3.107 0.001906 **
## X32         -2.52555    0.10332 -24.443  < 2e-16 ***
## X33          2.24767    0.16960  13.252  < 2e-16 ***
## X34          0.71753    0.17060   4.206 2.66e-05 ***
## X35          0.51165    0.36224   1.412 0.157897
## X36          0.77798    0.11678   6.662 3.08e-11 ***
## X37         -1.13305    0.20606  -5.499 4.07e-08 ***
## X39         -1.21764    0.09525 -12.784  < 2e-16 ***
## X40          0.72548    0.10513   6.900 6.01e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.04 on 3964 degrees of freedom
## Multiple R-squared:  0.9395, Adjusted R-squared:  0.939
## F-statistic:  1759 on 35 and 3964 DF,  p-value: < 2.2e-16
```

We observe that the model should exclude several variables that are not statistically significant. Even so, it achieves a good R^2. We also find a low p-value, rejecting the fundamental hypothesis test, i.e., rejecting that all model coefficients are equal to 0.

4.2 RESIDUAL ANALYSIS

```
par(mfrow=c(2,2))
plot(lm.fit)
```



The residuals appear consistent with normality.

4.3 MULTICOLLINEARITY ANALYSIS

```
vif(lm.fit)
```

```
##           X1           X2           X3           X4           X5           X6
##   408.490014   831.520339   192.849162  4234.265378  2104.713813  3822.684838
##           X7           X8           X9          X10          X11          X13
## 11111.756508  9918.796193   534.449679   788.812639    11.689165    48.020577
##          X14          X15          X16          X17          X19          X20
##    47.905111    20.911565    31.960974    19.108172    49.244073    22.872720
##          X21          X23          X24          X25          X26          X27
##    24.898027    46.207769    29.216850    51.113171    24.107272    57.028605
##          X29          X30          X31          X32          X33          X34
##    49.714604     9.439884     8.265261     7.132516    30.533898    16.536443
##          X35          X36          X37          X39          X40
##    41.421214     7.823699    44.246408     8.056928     8.011538
```

There is extreme multicollinearity, which indicates that this model cannot be implemented because it fails the assumptions. Recall that values above 5 are considered evidence of multicollinearity, and here the VIFs are highly disproportionate.

```r
det(vcov(lm.fit))
```

```
## [1] 3.518843e-73
```

The determinant of the design covariance matrix is extremely small, which would make the linear model "blow up".

5 REGULARIZATION MODEL

Apply elastic-net regularization with the glmnet library for alpha values: 0, 0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95, 1. Choose the best model by cross-validation and compare the models obtained.

5.1 MODEL CONSTRUCTION

We prepare the design matrix and the response vector.

```r
mx=as.matrix(datos[,2:ncol(datos)])
my=as.matrix(datos[,1])
```

We build cross-validated models over 12 α values and the default 100 λ values, for up to 1200 models as an upper bound.

```r
alphas <- c(0, 0.05, 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85, 0.95, 1)

cv_models <- lapply(alphas, function(a) {
  set.seed(123) # for reproducibility
  cv.glmnet(mx,my,keep=TRUE, alpha = a)
})
```

5.2 MODEL SELECTION

We use the 1*se* rule to select the best model for each *alpha*.

```r
cv_1se <- function(cv_models, alphas){
cv_errors_1se <- sapply(cv_models, function(model) {
  model$cvm[which(model$lambda == model$lambda.1se)]
})
cv_lambda_index <- sapply(cv_models, function(model) {
  which(model$lambda == model$lambda.1se)
})
cv_lambda_value <- sapply(cv_models, function(model) {
  model$lambda[which(model$lambda == model$lambda.1se)]
})

df<- data.frame(
  cvm = cv_errors_1se,
  lambda_index = cv_lambda_index,
  lambda = cv_lambda_value,
  alphas = alphas
```

```
)
return(df)
}
df=cv_1se(cv_models,alphas)
df
```

```
##         cvm lambda_index     lambda alphas
## 1  4.753534           98 0.65940100   0.00
## 2  4.329838           75 0.11206565   0.05
## 3  4.320376           69 0.06527930   0.15
## 4  4.322006           67 0.04717749   0.25
## 5  4.322504           66 0.03698374   0.35
## 6  4.314647           66 0.02876513   0.45
## 7  4.322476           65 0.02582975   0.55
## 8  4.318053           65 0.02185594   0.65
## 9  4.326425           64 0.02078862   0.75
## 10 4.323358           64 0.01834290   0.85
## 11 4.321695           64 0.01641207   0.95
## 12 4.321013           64 0.01559147   1.00
```

Note that the CVM is very similar across all α values.

We now check which α yields the best results in terms of mean squared error.

```
best_alpha_model_index <-which(df$cvm == min(df$cvm))
best_alpha_model_params <- df[best_alpha_model_index,]
best_alpha_model_params
```

```
##        cvm lambda_index     lambda alphas
## 6 4.314647           66 0.02876513   0.45
```

The best model has $\alpha = 0.45$.

```
set.seed(123)
best_alpha_model = cv.glmnet(mx,my,keep=TRUE, alpha = 0.45)
```

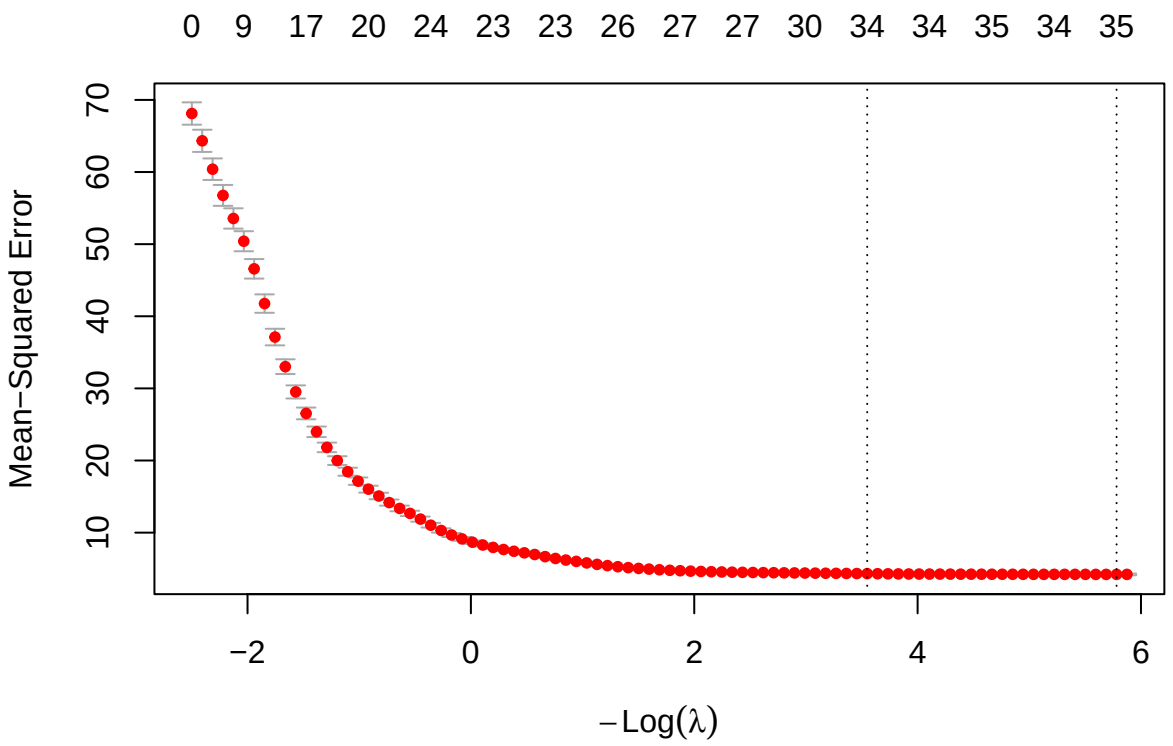
We now inspect some features of the best model.
We find a suitable value for λ.

```
nCVopt = which(best_alpha_model$lambda == best_alpha_model$lambda.1se)
best_alpha_model$lambda[nCVopt]
```

```
## [1] 0.02876513
```

We plot the information about λ.

```
plot(best_alpha_model)
```




```
best_alpha_model$glmnet.fit
```

```
##
## Call:  glmnet(x = mx, y = my, alpha = 0.45)
##
##    Df  %Dev  Lambda
## 1   0  0.00 12.1700
## 2   3  5.69 11.0800
## 3   6 11.48 10.1000
## 4   6 16.82  9.2030
## 5   8 21.51  8.3850
## 6   9 26.14  7.6400
## 7  10 31.69  6.9620
## 8  14 38.90  6.3430
## 9  16 45.74  5.7800
## 10 16 51.75  5.2660
## 11 16 56.87  4.7980
## 12 17 61.23  4.3720
## 13 18 64.97  3.9840
## 14 18 68.13  3.6300
## 15 19 70.79  3.3070
## 16 19 73.06  3.0130
## 17 20 74.98  2.7460
## 18 20 76.61  2.5020
## 19 20 77.98  2.2800
```

```
## 20 22 79.35  2.0770
## 21 22 80.54  1.8930
## 22 23 81.54  1.7240
## 23 24 82.71  1.5710
## 24 24 83.95  1.4320
## 25 24 85.01  1.3040
## 26 24 85.93  1.1890
## 27 23 86.71  1.0830
## 28 22 87.38  0.9868
## 29 22 87.95  0.8991
## 30 23 88.44  0.8192
## 31 23 88.87  0.7465
## 32 23 89.23  0.6801
## 33 24 89.54  0.6197
## 34 25 89.88  0.5647
## 35 24 90.32  0.5145
## 36 23 90.69  0.4688
## 37 23 91.00  0.4272
## 38 23 91.27  0.3892
## 39 26 91.59  0.3546
## 40 26 91.89  0.3231
## 41 26 92.14  0.2944
## 42 26 92.36  0.2683
## 43 26 92.55  0.2444
## 44 26 92.70  0.2227
## 45 27 92.84  0.2029
## 46 27 92.98  0.1849
## 47 27 93.09  0.1685
## 48 27 93.18  0.1535
## 49 27 93.26  0.1399
## 50 27 93.32  0.1274
## 51 27 93.38  0.1161
## 52 27 93.43  0.1058
## 53 27 93.47  0.0964
## 54 27 93.50  0.0878
## 55 27 93.53  0.0800
## 56 27 93.56  0.0729
## 57 28 93.58  0.0664
## 58 29 93.60  0.0606
## 59 29 93.63  0.0552
## 60 30 93.65  0.0503
## 61 30 93.67  0.0458
## 62 32 93.70  0.0417
## 63 33 93.72  0.0380
## 64 33 93.75  0.0346
## 65 34 93.76  0.0316
## 66 34 93.78  0.0288
## 67 34 93.81  0.0262
## 68 35 93.83  0.0239
## 69 35 93.85  0.0218
## 70 34 93.86  0.0198
## 71 34 93.87  0.0181
## 72 34 93.87  0.0165
## 73 34 93.88  0.0150
```

```
## 74 34 93.89  0.0137
## 75 34 93.89  0.0124
## 76 35 93.90  0.0114
## 77 35 93.91  0.0103
## 78 35 93.91  0.0094
## 79 35 93.92  0.0086
## 80 35 93.92  0.0078
## 81 34 93.93  0.0071
## 82 34 93.93  0.0065
## 83 34 93.93  0.0059
## 84 34 93.93  0.0054
## 85 34 93.94  0.0049
## 86 34 93.94  0.0045
## 87 35 93.94  0.0041
## 88 35 93.94  0.0037
## 89 35 93.94  0.0034
## 90 35 93.94  0.0031
## 91 35 93.94  0.0028
```

```
coef(best_alpha_model)
```

```
## 36 x 1 sparse Matrix of class "dgCMatrix"
##              lambda.1se
## (Intercept)  0.17294659
## X1           1.39255328
## X2           1.91389912
## X3          -0.18966177
## X4           0.06531778
## X5           0.98759369
## X6           2.46083843
## X7           0.47766097
## X8           0.12198935
## X9           0.88942337
## X10         -1.14350466
## X11          0.65295770
## X13          0.67450088
## X14          .
## X15         -0.38998129
## X16          0.28535873
## X17          1.03311448
## X19          0.03317793
## X20          2.15607404
## X21         -0.20414991
## X23          0.35219429
## X24         -1.77411444
## X25         -0.06594692
## X26         -2.17720229
## X27         -1.69920729
## X29         -1.54352823
## X30          3.97812194
## X31          0.77967773
## X32         -2.30746007
## X33          1.66637917
## X34          0.28614894
```

```
## X35         -0.87236841
## X36          0.82591280
## X37         -0.22641252
## X39         -0.83374838
## X40          0.70542968
```

Although $\lambda = 0.02876513$ is optimal under the 1se rule, a larger λ may be preferable here; for example, 0.0729 still keeps an R^2 of about 93% but with 27 variables instead of 34.

Finally, we make a prediction with the 1se model.

```
mxpred=mx[1001:1010,]
predict(best_alpha_model, mxpred,   s = best_alpha_model$lambda.1se, type=c("link"))
```

```
##       s=0.02876513
##  [1,]     184.4686
##  [2,]     177.2767
##  [3,]     148.8762
##  [4,]     198.9551
##  [5,]     175.0484
##  [6,]     185.0120
##  [7,]     175.5142
##  [8,]     162.7461
##  [9,]     170.2977
## [10,]     176.1462
```